TECHNICAL EVALUATION · 2 NOVEMBER 2026

NUR FOOD POS

Staging Verification
& Final Technical Audit

Prepared for
Emmanuel Varinot
Managing Director, NEVIE-GLOBAL™

Source review and isolated testing
Staging deployment not verified

NUR FOOD POS

Staging Verification & Final Technical Audit

Prepared for: Emmanuel Varinot, Managing Director, NEVIE-GLOBAL™ **Assessment date:** 2 November 2026 **Project:** Nur Food POS (`food_software`) **Assessment type:** Source audit, implementation review and isolated automated tests **Staging status:** Not deployed; no Vercel project or backend infrastructure was accessible to this assessment.

1. Executive Summary

This continuation adds a company/branch directory, database tenant constraints, controlled tenant provisioning, company-scoped branch administration and database-backed manual payment transaction records. Existing order totals, inventory movements and reports remain server-backed. The frontend design was not changed.

The local implementation and isolated test evidence is positive: **576 frontend assertions passed, 22 Laravel tests with 127 assertions passed, API proxy checks passed, and the Vite production build succeeded.** Cross-tenant API tests cover orders, products, inventory, shifts and reports. Tenant provisioning and branch authorization are also tested.

There is no verified staging deployment. The connected Vercel account returned no accessible project matching `food-software`, this checkout has no `.vercel/project.json`, and no staging backend/database credentials or hosting environment were supplied. Therefore no live URL, database persistence after service restart, browser/API workflow, backup/restore test, or real tenant penetration test can be reported. The product remains an **alpha prototype / demonstration MVP**, not production-ready.

2. Previous Audit Comparison

The previous report identified a static Vercel frontend disconnected from Laravel, local browser storage as business-data authority, incomplete tenant model, insecure local authentication assumptions, fabricated sample panels and unverified printer state. Source work in the preceding iteration connected frontend operations to a same-origin Laravel proxy and added database order/catalog/inventory/shift/report endpoints, Sanctum sessions, and server calculations. This continuation adds tenant directory/constraints, branch administration, secure CLI tenant provisioning, and manual payment transaction rows.

Previous test baseline: 576 frontend assertions; 19 Laravel tests / 101 assertions. Current rerun: 576 frontend assertions; 22 Laravel tests / 127 assertions. No source was pushed and no production data was modified.

3. Implemented Improvements

- Added `companies` and `branches` tables with company relationships and backfilled existing tenant identifiers before applying constraints.

- Added composite company/branch foreign keys for users, shifts, orders, catalog and inventory movements. Mismatched tenant/branch pairs cannot be inserted into those tables.
- Added company-scoped `GET /api/branches` and permission-checked `POST /api/branches`.
- Added guarded `php artisan pos:provision-tenant` for a company, initial branch and first admin. It requires explicit confirmation with `--force` in production and hashes the PIN; demo seeding remains blocked in production.
- Added `pos_payment_transactions`; every order creates manual transaction rows, and split tender stores each method separately. This does not imply gateway authorization or settlement.
- Added staging environment templates at `.env.staging.example` and `backend/.env.staging.example` and documented safe setup and restore rehearsal steps.
- Preserved existing frontend layout and POS navigation.

The migration and provision command have been exercised against the test suite's in-memory SQLite database. They have not been applied to a live staging or production database.

4. Company and Branch Architecture

`companies.id` is the tenant identifier. `branches` holds company, branch code, name, timezone and active status; the company/code pair is unique. Existing `company_id` and `branch_id` values are retained and backfilled. Business tables reference `(company_id, branch_id)` to `(branches.company_id, branches.code)` so a valid branch code from one tenant cannot be paired with another company.

The provisioning command creates the initial admin account scoped to the newly created company and branch. Branch API creation always takes company identity from the authenticated user and does not accept a client-supplied company ID. Branch creation requires admin role or `branches_manage` permission.

Remaining gaps: no company self-service UI or platform-level tenant manager, limited relational constraints on product/category and order/line relationships, and no test against the target production database engine. Four-digit PIN identity is ambiguous across a larger multi-tenant directory and must be redesigned before broad commercial onboarding.

5. Authentication and Security

Laravel Sanctum persists access tokens; the frontend receives an HttpOnly `nurpos_session` cookie and Vercel proxy maps it to a bearer token. PIN attempts are IP-rate-limited. The provisioning CLI uses a salted PBKDF2 PIN hash, rejects a PIN already assigned to another user, and creates a random unusable password value.

Authentication remains four-digit PIN based. MFA, recovery, PIN rotation, account lifecycle controls, session/device administration and audit event log are absent. IP-only throttling does not provide sufficient identity assurance for internet-facing administrator access. These are launch blockers; no UI redesign was made to introduce a different login flow.

API tenant filters derive from authenticated user data. Tests cover attempts to read another company's orders, shifts, products, inventory and report results. A production-grade penetration test and deployed infrastructure controls review are still required.

6. POS Features

Area	Implementation evidence	Status
Cart rows, quantities, notes and VAT-inclusive display	`verify_pos.js`, 576 assertions	UI regression harness passes. Server rejects priced modifiers until authoritative modifier configuration exists.
Product/category catalog	`CatalogController`, branch-scoped APIs and tests	Persistent create/update/category reassignment; no persistent upload storage.
Orders	`OrderController`, `PosApiTest`	Persistent transactional orders, server-side price/tax/discount math, idempotency, status and cancellation. Latest 100 in history endpoint.
Branch/tenant data	Tenant migration and API feature tests	Database enforced for core business row tenant/branch pair; no live-system test.
Inventory	`InventoryController`, sale/cancel test	Tracked stock adjustment, movement history and low-stock query; purchase/receiving and full UI remain incomplete.
Shifts	`ShiftController`, feature tests	Scoped open/list/close and ownership rules. Audited cash close/reconciliation not persisted.
Reports	`ReportController`, feature tests	Persisted date, branch, product and tender summaries. Specialized damage/service/commission/internal-order panels remain unavailable, local or incomplete.
Customers	Source review	Fixed customer channels/labels only; not a CRM.

7. Order and Payment Operations

Order amounts are calculated from branch product records and inclusive VAT rates in Laravel. Order lines, stock changes, manual payment entries and the order are written under database transaction boundaries. Client order IDs support tenant-scoped idempotent retries. Cancellation is authorized, tenant-scoped, and restores tracked stock once.

Manual payment records are stored in `pos_payment_transactions` with method, amount, status `recorded_manual`, tenant, branch, order and recorder. Split tender records individual declared parts. No payment processor, terminal, authorization, capture, settlement, refund or chargeback handling exists. A manual row is not evidence of funds received. Unconfigured modifiers/addons are rejected.

8. Inventory and Reporting

Inventory tracks quantity, adjustments, low-stock threshold and movement entries. Order sale/cancellation behavior is tested. Product receiving, suppliers, purchase orders, wastage approval and inter-branch transfers are absent. Seeded catalog values do not represent verified physical stock.

Main report aggregation uses persisted orders and is filtered to authenticated company/branch. Specialized panels remain incomplete where no authoritative table/workflow exists. Shift close does not yet create audited cash-count/reconciliation entries.

9. Database Architecture

Laravel/Eloquent and additive migrations are in `backend/database/migrations/`. New migrations:

- `2026_11_03_000001_create_tenant_directory_and_constraints.php`
- `2026_11_03_000002_create_pos_payment_transactions_table.php`

Other persistent models include users/tokens, orders/lines, shifts, categories, products and stock movements. Isolated tests use SQLite in memory. No production-compatible database server or staging schema was supplied. Migration behavior on MySQL/MariaDB, including performance and lock impact, must be tested against a sanitized copy before rollout.

10. Frontend and Backend Integration

The static Vite frontend uses same-origin `/api/*`; handlers in `api/` forward an allowlist through `api/_proxy.js` to Laravel configured by `NURPOS_LARAVEL_API_ORIGIN`. Vite provides a local proxy. The proxy fails closed without a configured origin, prevents arbitrary routes and does not forward the browser cookie upstream.

The API integration is source- and test-verified, not remotely deployed. Templates provide the required frontend API origin and Laravel staging/database settings. Replace placeholders in service secret managers only; do not commit filled files.

11. Staging Deployment Evidence

Deployment result: Not deployed. **Staging URL:** None. **Evidence:** Vercel account search for `food-software` returned zero accessible projects. No `.vercel/project.json` is present. No Laravel host, database, credentials, monitoring service, or isolated staging data was available.

The source includes `vercel.json`, a static build configuration, API proxy, and environment templates. These are deployment preparation, not staging deployment evidence. No production deployment or source push occurred.

12. Test Results

Check	Result	Limits
`node verify_pos.js`	576 passed, 0 failed	UI/business-rule harness; network/API calls are mocked.
`node verify_api_proxy.js`	Passed	Config fail-closed, route allowlist, cookie handling and no-store behavior.
`php artisan test --testdox`	22 passed, 127 assertions	SQLite in-memory feature tests; includes tenant constraints, branch administration/provisioning, cross-tenant reads, split manual tenders, orders, stock and reports.
`npm run build`	Passed	Vite 5.4.21. Warnings remain for classic script tags and deprecated CJS API.
PHP/JS syntax	Passed	All changed backend PHP and relevant JS parsed successfully.
Dependency audit	Unknown	npm and Composer registries were unreachable in this environment.
Browser/responsive and staging integration	Not run	No deployable staging environment or valid credentials.
Backup restoration	Not run	No staging database/backup was provided.

13. Backup and Recovery

No automated backup schedule, retention, off-site copy, RPO/RTO or restore proof exists. `backend/README.md` now includes MySQL/MariaDB logical backup guidance and an isolated restore rehearsal checklist. It requires a protected client configuration and encrypted off-host copy, then validation of migration version, tenant/branch counts, orders, line/payment rows and API reads. No database was backed up or restored in this assessment. Use forward migrations for production schema changes unless a reviewed, rehearsed rollback plan is approved.

14. External Services and Operating Costs

Vercel static hosting and an independently hosted Laravel API/database are required. An earlier illustrative low-traffic estimate was approximately $47.15/month before tax, domain, backups, monitoring, overages and staging; this is not a quote and provider prices may change. No staging infrastructure is currently connected. Payment provider fees, hardware and printer costs are unknown. Browser print is available; physical printer integration is not. ZATCA/e-invoicing compliance or certification is not established.

15. Remaining Limitations

1. Staging infrastructure is unavailable, preventing remote integration and persistence tests.
2. Four-digit PIN, no MFA/recovery/rotation/account lifecycle, and no admin audit log.
3. No platform-level tenant manager or company UI; tenant creation is controlled CLI only.

4. Some data relationships lack composite tenant foreign keys; migration engine compatibility needs staging validation.
5. No automated backup service or restore drill.
6. No payment gateway/terminal, refunds, settlement or certified e-invoicing.
7. Modifiers have no server-side catalog/pricing; full inventory purchasing and shift cash reconciliation are absent.
8. Specialized reports and CRM remain incomplete.
9. Vulnerability audit is unknown due registry/network failure.
10. No real browser/device, hardware or staging workflow acceptance run.
11. Copyright chain of title and exclusive transfer rights require legal confirmation.

16. Production Readiness Checklist

- [x] Company and branch entities and core tenant/branch constraints implemented.
- [x] Company-scoped branch admin API and interactive tenant provisioning command tested.
- [x] Cross-tenant order, shift, catalog, inventory and report access tests pass.
- [x] Manual payment transaction rows and split tender test pass.
- [x] Existing frontend regression checks pass; layout was not changed.
- [x] Vite build passes.
- [ ] Staging Vercel project and Laravel API/database provisioned and deployed.
- [ ] Run target-engine migration against sanitized copy and inspect locks/backfill.
- [ ] Test real browser → proxy → Laravel → persistent database end-to-end.
- [ ] Strong authentication, MFA/recovery, account controls and audit log.
- [ ] Backup automation and isolated restore drill.
- [ ] Online npm/Composer vulnerability review and remediation.
- [ ] Finish modifiers, operational inventory, payment settlement/refunds and shift reconciliation as required.
- [ ] Independent security and regulatory review; legal ownership clearance.

17. Acquisition Readiness Assessment

The source now has a stronger tenant/data foundation and can be handed to a technical team for continued development, subject to repository completeness, contributor history, license notices and legal ownership review. The proposed price or any monthly development arrangement is not approved by this report.

Independent operation is **not acquisition-ready**: the buyer must first provision staging and production infrastructure, run target database migrations safely, complete identity and audit controls, rehearse backup/recovery, perform online security checks, and verify required payment, hardware and regulatory workflows. The maturity classification remains **alpha prototype / demonstration MVP**, and the staging request remains unfulfilled due to unavailable Vercel/backend infrastructure access.